

CREATING A LINKED LIST AS A PREAMBLE TO TREES**PROBLEMS: (Each Question carries equal marks)**

1. According to the provided code, what is the default value assigned to pointers **pre** and **next** in the Node constructor?
 - A) 0
 - B) nullptr
 - C) garbage values
 - D) The address of the head
2. When the list is empty **head = nullptr**; which pointers are updated during an insertion?
 - A) Only the head
 - B) Only the tail
 - C) Both head and tail
 - D) Only the temp node
3. In the insert function, what is the purpose of the line **temp→pre = tail**;
 - A) It establishes a backward link to the previous node
 - B) It deletes the current tail node
 - C) It sets the tail to nullptr
 - D) It initializes the head pointer
4. Which line of code in the **showList** function is responsible for moving the traversal to the next element?
 - A) while (head != nullptr)
 - B) std::cout << head→data;
 - C) head = head→next;
 - D) Node *temp = new Node;
5. What is the output of the **main()** function execution?
 - A) 50 40 30 20 10
 - B) 10 20 30 40 50
 - C) 10 50
 - D) 30 30 30 30 30

6. In a **pop_front()** operation, if the list has more than one node, what must the **head→pre** pointer be set to after moving the head?
 - A) nullptr
 - B) The address of the old head
 - C) The address of the tail
 - D) It remains unchanged
7. Why are head and tail passed as **Node *&** in the insert function?
 - A) To prevent the function from changing the pointers
 - B) To allow the function to modify the actual head/tail pointers in main().
 - C) To save memory by not passing the actual data
 - D) Because C++ does not allow passing pointers by value
8. In the insert function provided in the code, why is the **new** keyword used when creating temp? What would be the consequence if memory was allocated locally without **new**?
9. Based on the struct Node definition, identify if this is a singly or doubly linked list. Support your answer by naming the specific element responsible for this classification.
10. In a **pop_back()** operation, why is it critical to update the next pointer of the node that becomes the new tail to nullptr?

```

void insert(Node *&head, Node *&tail, int value){
    Node *temp = new Node;
    temp->data = value;

    if (head == nullptr){
        head = temp;
        tail = temp;
    }
    else{
        tail->next = temp;
        temp->pre = tail;
        tail = temp;
    }
}

void showList (Node *head){
    while(head !=nullptr){
        std::cout<<head->data<<"\t";
        head = head->next;
    }
}

int main(){
    Node *head = nullptr , *tail = nullptr;

    insert (head, tail, 10);
    insert (head, tail, 20);
    insert (head, tail, 30);
    insert (head, tail, 40);
    insert (head, tail, 50);
    showList (head);
}

```